GHOSTMASK

Real-Time Face Blur & Identity Protection

Detect · Blur · Protect — all on-device, zero data stored

Python | MediaPipe | OpenCV | Privacy-First

THE PROBLEM

Privacy at Risk

Protest Footage

Publishing event recordings exposes individuals to identification without consent.

Street Media

Photographers and journalists risk violating privacy laws when identities are visible.

Data Leaks

Cloud-based tools store biometric data, creating breach risks and legal exposure.

MISSION

Detect every face. Blur every identity. Leave no data behind — all in real time, all on your device.

ARCHITECTURE

System Overview

INPUT
Webcam · Video · Image

DETECT
BlazeFace ML Model

BLUR
Pixel / Gaussian / Box

OUTPUT
Blurred Frame / File

File Responsibilities

face_detector.py
MediaPipe wrapper, bounding box extraction

processor.py
Pipeline orchestration, I/O, FPS loop

blur_engine.py
Pixel/Gaussian/Box blur + oval mask blending

main.py
CLI entry point, argparse routing

DETECTION

BlazeFace Neural Detection

Frame Capture

BGR frame read from camera or file via OpenCV

BGR → RGB

MediaPipe requires RGB input; single channel swap

BlazeFace Inference

ML model outputs normalized bounding boxes + confidence scores

Padding + Clamp

20% padding added; coords clamped to frame dimensions

Face List Output

Returns [{x1,y1,x2,y2,confidence}] for blur engine

Model Selection

DEFAULT

Model 0 — Short Range

Optimised for faces within ~2 m. Fastest inference, ideal for webcam and close-up footage.

CROWDS

Model 1 — Full Range

Handles faces up to ~5 m away. Slightly heavier but essential for surveillance or event footage.

⚡ *Key latency trick: CAP_PROP_BUFFERSIZE = 1 forces OpenCV to always return the freshest frame, eliminating queue lag.*

BLUR ENGINE

Three Modes, One Pipeline

DEFAULT

Pixel Mosaic

1. Crop face ROI
2. Downscale by block_size factor
3. Upscale with INTER_NEAREST
4. Result: blocky mosaic tiles

--block-size 15

SMOOTH

Gaussian Blur

1. Crop face ROI
2. Apply GaussianBlur kernel
3. Odd kernel size enforced
4. Result: frosted-glass look

--strength 55

FASTEST

Box Blur

1. Crop face ROI
2. Apply cv2.blur average filter
3. Smallest CPU footprint
4. Result: soft average blend

--strength 55

OVAL MASK — all three modes support an elliptical feathered blend: draw ellipse → Gaussian-blur the mask edges → alpha-composite blurred vs original. --shape oval (default)

VISUAL METHOD

Feathered Oval Masking

1. Crop ROI

Extract face bounding box from frame as a NumPy array slice

2. Blur ROI

Apply chosen blur algorithm to the cropped face region only

3. Ellipse Mask

Draw filled white ellipse on black canvas same size as ROI

4. Feather + Blend

GaussianBlur on mask edges, then alpha-composite into frame

blur_engine.py — oval mask blend (simplified)

```
mask = np.zeros((h_roi, w_roi), dtype=np.uint8)              # black canvas
cv2.ellipse(mask, (cx, cy), (cx-2, cy-2), 0, 0, 360, 255, -1)   # white ellipse
mask = cv2.GaussianBlur(mask, (k, k), 0)                     # feather edges
alpha = mask / 255.0                                         # 0–1 float mask
out = blur_roi * alpha + orig_roi * (1 - alpha)              # blend
```

USAGE

Three Ways to Use GhostMask

Webcam

- Live 30–60 FPS at 720p
- S key saves screenshot
- O / B toggle overlay & debug boxes
- CAP_BUFFERSIZE=1 eliminates queue lag

```
python main.py webcam --confidence 0.3 --block-size 20
```

Video File

- Reads any OpenCV-compatible format
- tqdm progress bar
- Optional live preview window
- Auto-names output file

```
python main.py video footage.mp4 --output safe.mp4
```

Still Image

- Single file in, single file out
- Supports JPG, PNG, WEBP
- Optional preview window
- Batch via shell loop

```
python main.py image photo.jpg --shape rect
```

PERFORMANCE

Speed & Latency

1–3ms	60fps	0 KB	100%
Detection / frame	Peak @ 720p i7	Data transmitted	On-device inference

Hardware Benchmarks

CPU	Resolution	Expected FPS
Intel i5 4-core	720p	30–45 fps
Intel i7 8-core	720p	45–60 fps
Apple M1 / M2	720p	50–70 fps
Raspberry Pi 4	480p	10–15 fps

Latency Tips

1. **Reduce resolution**
   640×480 gives ~2× FPS vs 1280×720
2. **Use Model 0**
   Short-range model is 30% faster on CPU
3. **Use --shape rect**
   Skip oval alpha-blend for raw speed
4. **Use box blur mode**
   cv2.blur is the cheapest operation

CLI REFERENCE

All Configuration Flags

--model 0|1
default: 0

Short-range fast vs. full-range crowd detection

--confidence
default: 0.4

Lower = catch more partial & occluded faces

--padding
default: 0.20

Fractional extra space added around face box

--mode
default: pixel

pixel · gaussian · box blur algorithm

--block-size
default: 15

Mosaic tile size in pixels (pixel mode only)

--strength
default: 55

Kernel size for gaussian/box (must be odd)

--shape
default: oval

oval (feathered ellipse) or rect (fastest)

--camera
default: 0

Camera device index for webcam mode

--width / --height
default: 1280×720

Capture resolution (lower = faster)

Privacy-critical (e.g. protest footage)? Use --confidence 0.25 --model 1 --padding 0.30 for maximum face coverage.

GHOSTMASK

Detect · Blur · Protect

Low Latency

BlazeFace + ROI-only processing keeps detection under 3 ms per frame

Partial Faces

0.4 confidence threshold + 20% padding catches occluded and profile faces

Oval Masking

Feathered elliptical blend — natural, clean, no harsh rectangle edges

Zero Data

Fully local. No storage, no network calls, no biometrics retained

Quick Start

pip install -r requirements.txt → python main.py webcam